Software Engineering Group
Heidelberg University

Prof. Dr. Sebastian Baltes
Tobias Reichel

Advanced Software Engineering (IASE)
Assignment 4

Automated Test-Input Generation (Fuzzing)

Submitted by:

Rohit Sonejee Matrikelnummer: 4756184

Thanh Tran Matrikelnummer: 4209012

Group 18

Summer Semester 2026

June 12, 2026

Declaration of GenAI Use

We, **Rohit Sonejee** (ID 4756184) and **Thanh Tran** (ID 4209012), declare that we used generative AI tools in the development of this assignment. Specifically, we used **Gemini** and **Claude**. Below is a transparent breakdown of what was done independently versus with AI assistance.

Parts Completed Independently

- Researching and understanding the theoretical distinctions between mutational, lexical, and grammar-based fuzzing paradigms.
- Designing the core architecture and implementing the grammar expansion logic in `GrammarFuzzer.expand` to accurately traverse and generate derivation trees in strict adherence to the provided EBNF syntax.
- Manually analyzing complex crash outputs, tracing execution paths, and identifying root causes (e.g., diagnosing the stack overflow triggered by deep nesting grammar payloads).
- Setting up the local testing environment, configuring Gradle tasks, and systematically verifying all implementations against the test suite to ensure valid fuzzing outputs.

AI-Assisted Parts

- **Gemini** & **Claude** – utilized as pair-programming aids for the basic character-level string mutators (`deleteRandomCharacter`, `insertRandomCharacter`, `flipRandomCharacter`, `repeatRandomCh`
- Received structural suggestions for the mutator selection loop in `MutationalFuzzer.kt` and generated a standard automation script to execute the binary and capture crash exit codes.
- Assisted with the general drafting and formatting of this LaTeX report.

Implementation Details

For this assignment, we built an automated test-input generator ("fuzzer") to find crashes in a small TOML parser. The project was divided into two main sub-exercises:

Exercise 4.1: Mutational Fuzzer

We implemented a mutational fuzzer that starts from a valid seed input and iteratively applies random edits.

- **Mutators:** In `Mutators.kt`, we implemented functions to apply small edits such as deleting a random character, inserting a random character from the alphabet, flipping a random low bit of a chosen character, and repeating a character.
- **Selection Logic:** In `MutationalFuzzer.kt`, we updated the `mutate` function to randomly pick and apply one of the mutators using the provided `Random` instance to ensure deterministic and reproducible runs.
- **Crash Discovery:** We ran our mutational fuzzer against the provided binaries (`toml_parser_win_x86_64` on Windows 11 x86_64) and successfully discovered two distinct crashes:
  1. **Exit Code -1073741819** (Access Violation)
  2. **Exit Code -1073740940** (Heap Corruption)

  The representative crashing inputs were accurately grouped and documented in `REPORT.md`.

Exercise 4.2: Grammar-Based Fuzzer

Since mutational fuzzers struggle to generate deeply nested structures like balanced brackets without breaking the syntax, we implemented a context-free grammar-based fuzzer.

- **Grammar Expansion:** We implemented the `GrammarFuzzer.expandNode` function to recursively expand pending EBNF grammar nodes based on their specific constructs (*Sequence, Choice, Optional, Repetition, Group*).
- **Deep Nesting Bug:** By executing the grammar fuzzer with a target recursive depth of 300, we forced the fuzzer to consistently expand recursive tree structures. This reliably pushed the parser into a deep recursive state, triggering a **Stack Overflow** crash.
- As documented in our report, a regular lexical or mutational fuzzer is extremely unlikely to hit this bug, as it cannot intelligently "remember" and maintain balanced, multi-level structural nesting on its own.

Test Verification

All required functionality has been implemented and tested. Below is the output of the `gradle test` command confirming that the 20 initially failing tests now pass successfully.

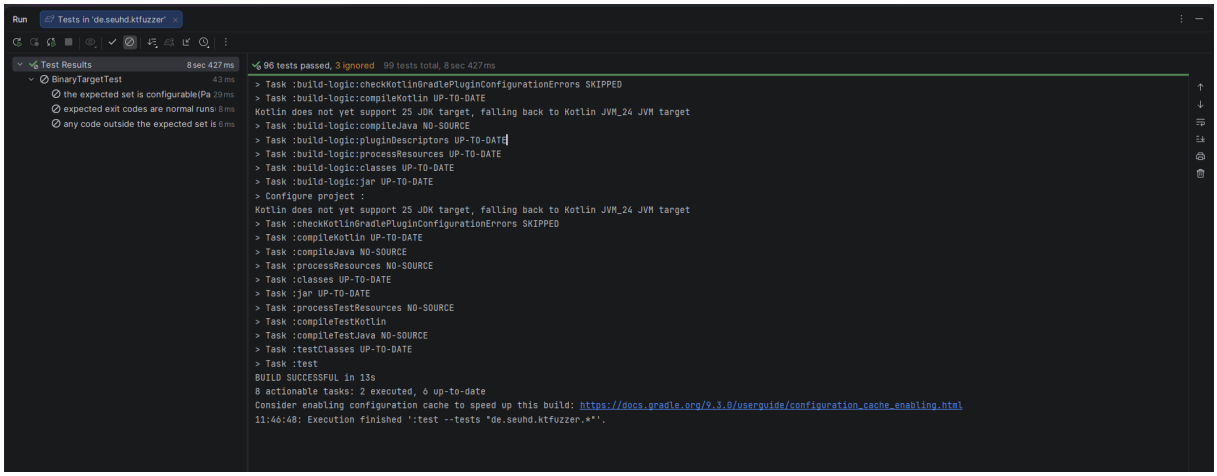

Figure 1: `gradle test` – All tests passing successfully.